IEEE Access
Multidisciplinary : Rapid Review : Open Access Journal

Received 25 October 2024, accepted 16 December 2024, date of publication 24 December 2024, date of current version 10 January 2025.

Digital Object Identifier 10.1109/ACCESS.2024.3522049

RESEARCH ARTICLE

A Reproducible Analysis of Sequential Recommender Systems

FILIPPO BETELLO[1], ANTONIO PURIFICATO[1], FEDERICO SICILIANO[1], GIOVANNI TRAPPOLINI[1], ANDREA BACCIU[1], NICOLA TONELLOTTO[2], AND FABRIZIO SILVESTRI[1], (Member, IEEE)

[1]Department of Computer, Control and Management Engineering, Sapienza University of Rome, 00185 Rome, Italy
[2]Information Engineering Department, University of Pisa, 56122 Pisa, Italy

Corresponding authors: Filippo Betello (betello@diag.uniroma1.it), Antonio Purificato (purificato@diag.uniroma1.it), and Federico Siciliano (siciliano@diag.uniroma1.it)

This work was supported in part by the European Union–NextGenerationEU through the Project FAIR under Grant PE0000013, through the Project SERICS under Grant PE00000014, and through the Project SoBigData.it under Grant IR0000013; in part by the Neural Reasoning Over Open Data (NEREO) Project funded by the Italian Ministry of Education and Research (PRIN) under Grant 2022AEFHA; and in part by the Project SEEDS: Sustainable Ecosystems in Evolving Digital Societies by Sapienza University of Rome–Progetti Dipartimentali.

ABSTRACT Sequential Recommender Systems (SRSs) have emerged as a highly efficient approach to recommendation systems. By leveraging sequential data, SRSs can identify temporal patterns in user behaviour, significantly improving recommendation accuracy and relevance. Ensuring the reproducibility of these models is paramount for advancing research and facilitating comparisons between them. Existing works exhibit shortcomings in reproducibility and replicability of results, leading to inconsistent statements across papers. Our work fills these gaps by standardising data pre-processing and model implementations, providing a comprehensive code resource, including a framework for developing SRSs and establishing a foundation for consistent and reproducible experimentation. We conduct extensive experiments on several benchmark datasets, comparing various SRSs implemented in our resource. We challenge prevailing performance benchmarks, offering new insights into the SR domain. For instance, SASRec does not consistently outperform GRU4Rec. On the contrary, when the number of model parameters becomes substantial, SASRec starts to clearly dominate all the other SRSs. This discrepancy underscores the significant impact that experimental configuration has on the outcomes and the importance of setting it up to ensure precise and comprehensive results. Failure to do so can lead to significantly flawed conclusions, highlighting the need for rigorous experimental design and analysis in SRS research. Our code is available at https://github.com/federicosiciliano/easy_lightning.

INDEX TERMS Recommendation, sequential recommendation, reproducibility, replicability, resource.

I. INTRODUCTION

Recommender Systems (RSs) have been pivotal in enhancing user experience for many years by providing personalized recommendations across numerous online platforms [1]. Among these, Sequential Recommender Systems (SRSs) have received significant attention for their ability to consider the temporal dynamics inherent in user preferences to predict the next item a user will engage with [2]. Initially, SRSs used Markov chains to model user behaviour [3], but faced challenges in capturing complex dependencies in extended user sequences [4]. The emergence of neural networks, especially recurrent structures [5], [6], marked a significant shift in this paradigm. Furthermore, the recent successes of transformer models have inspired the development of novel SRS architectures [7], [8].

Despite these advancements, the reproducibility[1] of these models remains an open problem [9], [10]. This issue is caused by several factors, mainly the lack of standardised benchmarks, uniform data pre-processing methods and consistent model implementations [11], [12], [13],

The associate editor coordinating the review of this manuscript and approving it for publication was Yin Zhang.

[1]We refer to the definition of *reproducibility* of the ACM Artifact Review and Badging, version 1.1, available online at https://www.acm.org/publications/policies/artifact-review-and-badging-current

© 2024 The Authors. This work is licensed under a Creative Commons Attribution 4.0 License.
For more information, see https://creativecommons.org/licenses/by/4.0/

which exacerbate the problem of reproducibility. Researchers frequently adopt different configurations, including various choices of hyperparameters and evaluation metrics [10], [13], [14]. The variability of data processing methodologies across studies prevents direct comparability of results [15]. Differences in model implementations make it difficult to conduct a reliable and comparable evaluation of SRSs [14], [16]. Finally, models are not consistently compared to the original settings described in the respective papers, introducing further ambiguity and preventing the establishment of a clear performance hierarchy [16]. The existence of different rankings of the proposed methods in each individual research paper hinders our ability to accurately assess real progress in the sequential recommendation domain [17].

We present EasyRec, a novel library dedicated to Sequential Recommender Systems (SRSs), designed to simplify data preprocessing and streamline model implementation. EasyRec provides a set of datasets and models, giving researchers and practitioners with an easy-to-use platform to facilitate experimentation and accelerate progress in the field of SRSs. Moreover, we conduct extensive experiments on multiple benchmark datasets, systematically comparing different SRSs implemented within our framework. This allows for a fair and unbiased evaluation, shedding light on the impact of crucial factors such as the length of the input sequence and the number of parameters.

The results of our study challenge existing performance benchmarks and provide new insights into the field of sequential recommendation. Surprisingly, our results show that SASRec does not consistently outperform GRU4Rec. However, an interesting shift occurs when the number of model parameters increases substantially: SASRec begins to clearly dominate all other SRSs.

Our contributions can be summarised as follows:

- Provision of a comprehensive code resource, which allows different data pre-processing methods, facilitates the implementation of various SRSs, and ensures full experimental reproducibility.
- Extensive experiments on multiple datasets and models, reducing biases in performance comparisons.
- In-depth analysis of the results, providing new insights into different models and the impact of key hyperparameters;
  1) Contrary to the prevailing notion in many articles, GRU4Rec consistently shows strong performance, often outperforming other SRSs.
  2) Performance gains for transformer-based models are more pronounced at higher embedding dimensions compared to GRU-based models, leading to SASRec outperforming all other models.
  3) The optimal input sequence length is related to the average user sequence length of each dataset, highlighting the importance of dataset-specific considerations.
  4) Transformer-based models exhibit exponential growth in size with increasing embedding dimensions, but performance improvements do not scale similarly. In addition, emissions are not only correlated with model size, but are significantly higher for models that require longer training times.

II. RELATED WORK

A. SEQUENTIAL RECOMMENDER SYSTEMS

Markov chain-based techniques [3] are among the first approaches in SRSs, but they struggle to capture complex dependencies within long sequences [4].

As a result, neural networks have recently emerged as the leading models for sequential recommendation. Recurrent Neural Networks [18] were among the first architectures introduced for this task. For instance, GRU4Rec [6] is a recurrent neural network using Gated Recurrent Units (GRUs) [5]. Another notable approach using GRUs is the Neural Attentive Recommendation Machine (NARM) [19], which features a hybrid encoder-decoder architecture with a custom attention mechanism. This mechanism captures both local and global preferences in user-item interactions.

Advances in natural language processing have led to attention-based models. Self-Attentive Sequential Recommendation (SASRec) [8] uses self-attention to discern the relevance of each item in the user's sequence. BERT4Rec [7] instead exploits the Bidirectional Encoder Representations for Transformer (BERT) architecture [20]. Its bidirectional self-attention enables the model to obtain more accurate representations of user behaviour, improving recommendation performance. To avoid information leakage, BERT4Rec is trained on the Cloze task, i.e. predicting randomly masked items by jointly considering the left and right context in the input sequence.

A recent addition to the landscape of SRSs is CORE [21], which also integrates an attention mechanism. CORE offers two variants: one with custom attention computation and another using standard attention mechanisms. Both allow the model to weigh the contribution of each item in the input sequence, further refining the recommendation process.

B. REPRODUCIBILITY IN SRSS

Despite being a long-standing issue, reproducibility in SRSs has not received sufficient attention. Ferrari Dacrema et al. [12] sheds light on the problem, showing that only a fraction of top-quality work can be faithfully reproduced [22]. In support of this observation, Hidasi et al. [14] show that non-original implementations often produce inferior results than the original papers. Even implementations found in common and widespread libraries, such as RecBole [23], present this issue [16] and fail to faithfully reproduce reported results. Furthermore, even when results are reproducible, the lack of standardisation in faulty experimental setups (e.g. varying loss functions, user' sequence length, etc.) [15] makes it impossible to establish a clear hierarchy between different models [16], [24], [25], [26]

and to establish if we are really making progress in the SRS domain [9]. This inconsistency casts doubt on real progress in the SRS field [9].

Our work addresses these challenges by providing a comprehensive code resource using current versions of prominent Python libraries such as PyTorch and PyTorch Lightning. This framework ensures a faithful reproduction of the original paper results, demonstrating the robustness of our implementation. Furthermore, we conduct a rigorous comparative study under consistent conditions, allowing for a nuanced analysis of the hierarchy between SRSs. This approach contributes to a more reliable comparison process than current studies provide.

III. BACKGROUND

The primary goal of sequential recommendation is to predict the next interaction within a sequence. Let's consider a collection of n users, denoted as $\mathcal{U} \subset \mathbb{N}^+$ and a corresponding set of n items denoted as $\mathcal{I} \subset \mathbb{N}^+$. Each user $u \in \mathcal{U}$ is characterised by a temporally ordered sequence $S_u = [s_1, \ldots, s_{L_u}]$, where $s_i \in \mathcal{I}$ represents the i-th item the user has interacted with. Here, $L_u > 1$ denotes the sequence length, which may vary from user to user.

A SRS, denoted as $\mathcal{M}$, takes as input the sequence up to the L-th item, expressed as $S_u^L = [s_1, \ldots, s_L]$, and attempts to recommend the next item, s_{L+1}. The output of the recommendation, represented as $r^{L+1} = \mathcal{M}(S_u^L) \in \mathbb{R}^m$, is a score distribution over all items. This distribution is used to generate a ranked list of items representing the most likely interactions for user u at step $L+1$. The following item s_{L+1}, is regarded as the *positive* item, meaning that it is the item to be predicted. In contrast, all other items that should not be predicted at step $L+1$ are labelled as *negative* items.

A. INPUT SEQUENCE LENGTH

The choice of input sequence length L is a crucial hyperparameter: opting for short sequences may provide insufficient information to the model, while excessively long sequences may inject irrelevant information. Given the varying number of interactions per user L_u, it is possible that L_u is shorter than the chosen L. In such cases, the sequence is padded at the beginning with a dedicated identifier $pad \notin \mathcal{I}$, typically 0. Consequently, if $L_u < L$, the sequence is modified to $S_u^L = [pad, \ldots, pad, s_1, \ldots, s_{L_u}]$, with the number of padding instances equal to $L - L_u$. This approach ensures consistency in sequence lengths and allows for effective use of the model while accommodating variations in user interaction lengths.

B. ITEM EMBEDDING

The representation of items by identifiers alone lacks essential information about their inherent properties. To provide these representations with meaningful content, it is common practice to use a lookup table $\mathcal{E} : \mathcal{I} \rightarrow \mathbb{R}^d$. This table serves as a repository for embeddings, i.e. meaningful vectors that reside in a latent space with dimensionality d. This approach facilitates the transformation of item identifiers into rich, informative vectors that provide a nuanced and comprehensive representation of the main features of each item.

IV. IMPLEMENTATION FRAMEWORK AND EXPERIMENTAL SETUP

A. RESOURCE

Our EasyRec library, is designed with several key features, listed below, that significantly enhance its utility for researchers and practitioners alike.

1) REPRODUCIBILITY

In our approach, we address two main challenges to improve the reproducibility of our experiments. First, we follow best practices for experimental seeding to ensure consistent and reliable results across multiple runs. Second, we specify the versions of the required Python packages, to prevent errors due to version mismatches, ensuring that our pipeline remains stable and predictable across different setups. While the exact reproducibility of results is affected by non-negligible hardware and software factors, such as Python library versions and underlying hardware, we document all settings in our code repository.[2]

2) EXTENSIBILITY

The resource is not restricted to the datasets, models or experimental setups used in this work. To include new datasets, it is sufficient to process them to adhere to the same format used in this work. For models, they can be easily added as PyTorch Module objects, with adjustments made in a configuration that specifies their input and output. The experimental setup is easily modifiable through a YAML file containing the entire configuration, enabling experimentation with scenarios not considered here. Further extensibility, described in detail in the code repository[2], includes the use of different loss functions, optimisers and more.

3) MODEL IMPLEMENTATIONS

A common drawback of many existing implementations of popular SRSs is their overly complex and lengthy code, often using custom implementations of standardised methods. This makes it difficult to understand the model's operation and to assess its compliance with the mathematical principles underlying the methodology. To address this issue, we undertook a comprehensive rewrite of these models, condensing them down to the most essential lines of code. Using PyTorch's pre-implemented methods, such as GRU and Attention, we were able to improve their clarity and comprehensibility. Additionally, PyTorch implementations are widely adopted, standardized, and rigorously maintained, reducing the likelihood of errors and ensuring timely resolution of any issues. By aligning with established PyTorch practices, our code stands to benefit from ongoing improvements in the PyTorch

[2]https://github.com/federicosiciliano/easy_lightning

ecosystem. In addition, our library supports wandb,[3] which can be used to track experiments and perform hyperparameter optimization.

4) RUNNING EXPERIMENTS

The software environment can be reproduced by taking the Python library versions specified in our library requirements. A YAML file contains the experiment configuration, specifying all hyperparameters. A Python script reads this configuration and streamlines model training. Detailed information can be found in the code repository[2].

B. DATASETS

Our dataset collection includes the following:

- Amazon[4]: These datasets consist of product reviews collected from Amazon.com by McAuley et al. [27]. The data are organized into distinct datasets based on Amazon's primary product categories. For our study, we focus on the "Beauty" category (Beauty).
- Foursquare[5]: These datasets contain check-ins collected over a period of approximately ten months [28]. We use the New York City (FS-NYC) and Tokyo (FS-TKY) versions.
- MovieLens[6]: The MovieLens dataset [29] is widely recognized as a benchmark for evaluating recommendation algorithms. We utilize three versions: MovieLens 20M (ML-20M), MovieLens 1M (ML-1M) and MovieLens 100k (ML-100k).

The statistics of these datasets are presented in Table 1.

TABLE 1. Dataset statistics after pre-processing; users and items not having at least 5 interactions are removed. Avg. and Med. refer to the average and median of $\frac{\text{Actions}}{\text{User}}$, respectively.

Name	Users	Items	Interactions	Density	Avg.	Med.
Beauty	1,274	1,076	7,113	0.519	5,58	5
FS-NYC	1,083	9,989	179,468	1.659	165	116
FS-TKY	2,293	15,177	494,807	1.421	215	146
ML-100k	943	1,349	99,287	7.805	105	64
ML-1M	6,040	3,416	999,611	4.845	165	96
ML-20M	138,493	18,345	19,984,024	0.787	144	68

1) DATA PRE-PROCESSING

Our pre-processing strategy follows established practices, such as treating ratings as implicit, i.e. use all interactions without considering the rating value, and removing users and items with fewer than 5 interactions [7], [8], [16]. For testing, as in [7] and [8], we keep the last interaction for each user, while for the validation set, the second to last action is retained. All remaining interactions contribute to the training set. Each observed user-item interaction is treated as a positive instance, and negative items are sampled from items not previously consumed by the user.

C. MODELS

We employ five sequential recommendation models in our experiments, each based on seminal papers widely cited in the literature. While a more detailed discussion of these models is provided in section II, here we offer a succinct overview of their differences:

- BERT4Rec [7]: This model is based on the BERT architecture, enabling it to capture complex relationships in user behaviour sequences through bidirectional self-attention.
- CORE [21]: it introduces an attention mechanism that enables the model to weigh the contribution of each item in the input sequence, enhancing recommendation accuracy.
- GRU4Rec [6]: This model utilizes GRUs to capture temporal dependencies in user-item interactions.
- NARM [19]: it also leverages GRUs, but incorporates a custom attention mechanism to capture both local and global preferences in user-item sequences.
- SASRec [8]: this model is characterized by its use of self-attention mechanisms, allowing it to discern the relevance of each item within the user's sequence.

It's crucial to distinguish between sequence-to-sequence models (GRU4Rec, SASRec, BERT4Rec) and sequence-to-item models (NARM, CORE). Sequence-to-sequence models analyze input sequences in detail, generating predictions at each step, addressing issues like vanishing gradients in long sequences [30]. They require fewer samples as each sequence generates multiple predictions, effectively increasing training capacity. Conversely, sequence-to-item models process the entire input sequence but only predict at the last step, reducing training capacity compared to sequence-to-sequence models. The original works [19], [21] likely counter this by using shorter input lengths, reducing the need for a large number of training samples. A schematic diagram illustrating the functioning of each model is provided in Fig. 1.

1) TRAINING

A common loss function for training SRSs [8] is Binary Cross Entropy (BCE) [31]. While some studies explore variations in training methods [6], [19], our primary focus is on evaluating the architecture itself, with the aim of isolating its capabilities from the effects of different loss functions.

We trained all models for 400 epochs, and for each model, we selected the checkpoint from the epoch where it achieved the highest performance on the validation set. We also utilize the same optimiser, batch size, number of negative items per training and testing instance, and other relevant parameters. By maintaining consistency in these aspects, we aim to eliminate potential confounding factors and facilitate a clear assessment of architectural differences. For ease of reproducibility and transparency, all relevant parameters and

[3] https://wandb.ai/site/
[4] https://jmcauley.ucsd.edu/data/amazon/
[5] https://sites.google.com/site/yangdingqi/home/foursquare-dataset
[6] https://grouplens.org/datasets/movielens

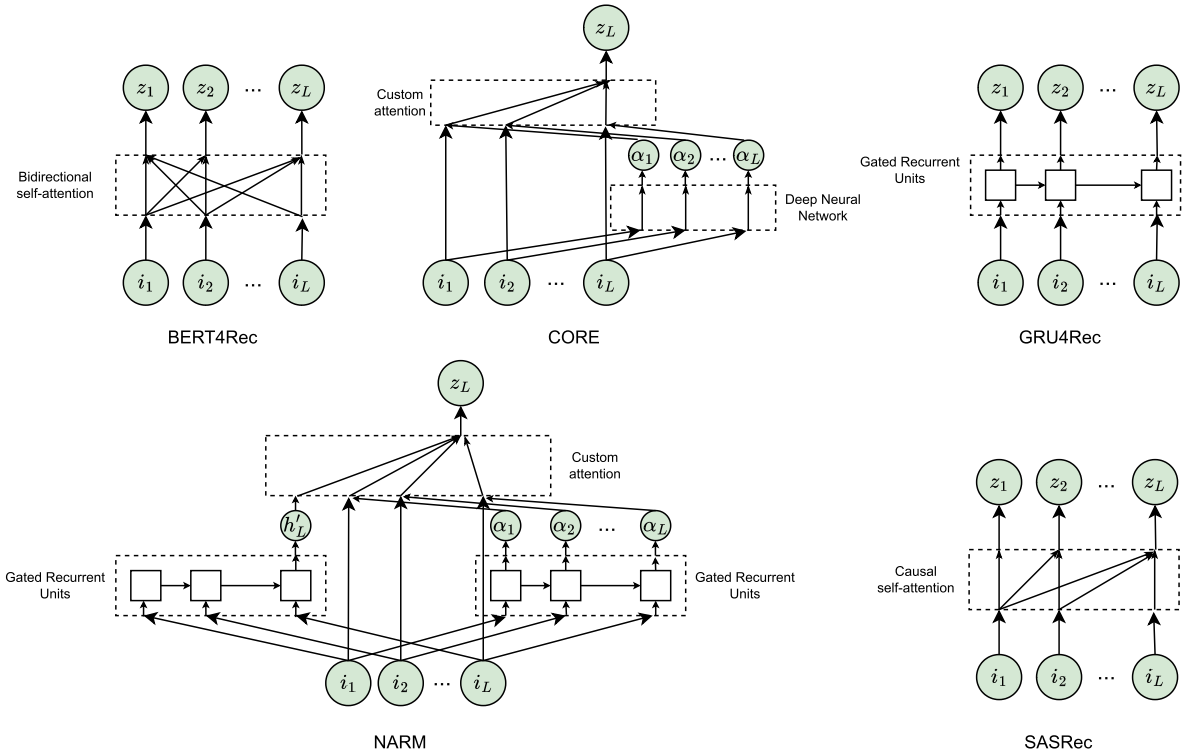


FIGURE 1. A visual depiction illustrating the core functionality of the considered SRSs. The input items $i_1, i_2, \ldots, i_L$ are processed to generate the final representations $z_1, z_2, \ldots, z_L$, which are utilized to generate predictions at steps $1, 2, \ldots, L$, respectively. Intermediate representations $h_1, h_2, \ldots, h_L$ are also present for some models.

configurations are carefully documented and made available in our code repository[2]. All experiments were performed on a single NVIDIA RTX A6000 with 10752 CUDA cores and 48 GB of VRAM.

D. METRICS

To evaluate the performance of sequential recommendation algorithms, we use four widely used metric, also common in Information Retrieval (IR): Precision, Recall, NDCG and MAP.

1) TRACKING EMISSIONS

In the Glasgow Agreement [32] climate change highlights the international commitment to reduce CO_2 emissions. This is especially important in the field of Artificial Intelligence, where training GPUs has a significant environmental impact. The energy consumption associated with these computational processes contributes significantly to CO_2 emissions, exacerbating climate change [33]. It is therefore our responsibility to raise awareness of this issue.

In this study, we use CodeCarbon[7] [34], a tool designed to track the power consumption of both CPUs and GPUs. This allows us to measure carbon dioxide equivalent (CO_2-eq), a widely accepted standard to monitor emissions of various greenhouse gases [35]. CO_2-eq facilitates the comparison of greenhouse gas emissions by converting quantities of different gases into an equivalent amount of CO_2, based on their respective global warming potentials.

E. COMPARISONS

The importance of our resource lies primarily in the fact that it fills a notable gap in the existing literature and libraries. In particular, Elliot [36] and DaisyRec [10] lack SRS models, while Cornac [37] and RecPack [38] provide only GRU4Rec among newer models. Although ReChorus [39] supports many models, it lacks one of the most cited ones (BERT4Rec) and modern ones. Conversely, RecBole [23] stands out as a comprehensive framework that is widely used in the field. However, it requires the creation of a subclass of its classes to develop new models or datasets. In contrast, our approach allows the extension of the PyTorch standard class for all new models. Furthermore, existing literature highlights suboptimal performance metrics, timing, and implementation quality in RecBole [14], [16]. Recbole's performance, evaluated on the same number of epochs, is compromised by performing many iterations per epoch due to their data augmentation method.[8] Moreover, they did not implement BCE loss, which is used in several works [8],

[7] https://codecarbon.io

[8] https://recbole.io/docs/recbole/recbole.data.dataset.sequential_dataset.html

TABLE 2. Comparison using MovieLens 1M dataset. RecBole^a represents a model trained until it matches our metrics, while RecBole^b refers to the model trained for the same time as ours.

Model	GRU4Rec			SASRec			BERT4Rec		
Implementation	*EasyRec*	*RecBole^a*	*RecBole^b*	*EasyRec*	*RecBole^a*	*RecBole^b*	*EasyRec*	*RecBole^a*	*RecBole^b*
NDCG@10	0.56	0.56	0.48	0.51	0.51	0.45	0.49	0.49	0.43
Recall@20	0.88	0.88	0.85	0.86	0.86	0.84	0.86	0.86	0.83
Time	28min	1330min	32min	38min	360min	48min	51min	220min	68min

as also indicated on the official GitHub repository,[9] so their implementations are not loyal to the originals. We changed their loss to be consistent with the original works. Table 2 shows the comparison between our implementation and that of RecBole using ML-1M and the three most common and widely used models in the literature: SASRec, GRU4Rec, and BERT4Rec, in terms of metrics and time. By RecBole^a, we refer to models that are tracked until they reach our metrics, while by RecBole^b we refer to models that are tracked for our own time. The experimental setup is the same as described in section V-A. We outperform RecBole in every model.

V. RESULTS

Our experiments aim to address the following research questions:

RQ1: What is the hierarchy of current SRSs and is it consistent with the existing literature?

RQ2: How does the performance of SRSs vary with changes in (a) embedding size and (b) input sequence length?

RQ3: Using a more appropriate comparison based on the total number of parameters, what insights can be gained about the hierarchy of the models?

RQ4: Is there a discernible trade-off between model performance and CO_2 emissions?

A. RQ1: BASE PERFORMANCE

In Table 3 we present the performance metrics of the reproduced models on the six datasets considered in this study. For the sake of experimental consistency, we maintain a uniform embedding size of 50 across all models. The input sequence lengths are set to 200 for FS-TKY, ML-20M, ML-1M, 100 for ML-100k and FS-NYC, and 50 for Beauty.

We verify the consistency of our results by comparing them to those reported in the original studies, although with some limitations due to differences in dataset selection between different papers. For overlapping datasets and similar experimental setups, we observe similar performance metrics, which supports the reliability of our implementation.

The first novel insight proposed by our paper is that our results do not confirm the conventional belief that SASRec consistently outperforms GRU [7], [8], [19], [21]. Contrary to this notion, GRU4Rec beats the competitors on the 3 MovieLens datasets.

[9]https://github.com/RUCAIBox/RecBole/issues/1667

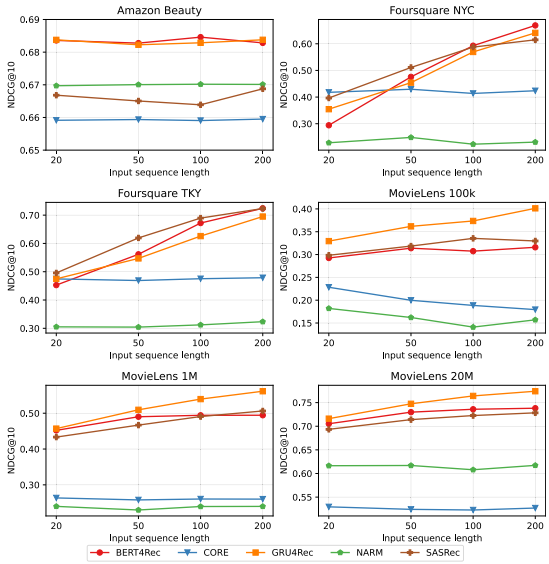


FIGURE 2. Effect of input sequence length on model performance, as measured by NDCG@10. Each plot shows the results of the five models on one dataset.

In particular, the exceptional performance of GRU4Rec in our experiments may be attributed to the quality of its implementation. As observed by Hidasi et al. [14], GRU4Rec inferior performance in other studies [7], [8], [19], [21] can be attributed to the quality of existing implementations being suboptimal. By using Torch's implementation of the GRU, our framework benefits from a high-quality implementation, which may contribute to the observed superior performance compared to other frameworks.

Furthermore, the performance of NARM and CORE is consistently below that of other models. As mentioned in section IV-C, this disparity can be attributed to the sequence-to-item training paradigm employed by these architectures, which differs from the sequence-to-sequence training employed by other models. Instead, they are able to achieve high results on datasets with short input sequences, such as Beauty, as evidenced by the best Precision and Recall @20. This finding is consistent with the observations of Hou et al. [21], which highlights the ability of CORE to effectively capture short-term user preferences.

B. RQ2(A): INPUT SEQUENCE LENGTH

Fig. 2 shows the results of a detailed investigation aimed at exploring the impact of sequence length on the model's

TABLE 3. Results of the proposed models in terms of Precision@K (P@K), Recall@K (R@K), NDCG@K and MAP@K, with K ∈ {10, 20}. **Bold** denotes the best model for a dataset by the metric in the main group, underlined the second best. † indicates a statistically significant result with p-value < 0.01.

Dataset	Model	P@10	R@10	NDCG@10	MAP@10	P@20	R@20	NDCG@20	MAP@20
Beauty	**BERT4Rec**	**0.0707**	**0.7072**	**0.6828**	0.1991	0.0362	0.7237	0.6848	**0.1233**
	CORE	0.0695	0.6954	0.6594	0.1926	0.0366	0.7316	0.6673	0.1199
	GRU4Rec	0.0704	0.7041	0.6823	**0.1992**	0.0360	0.7198	**0.6855**	**0.1233**
	NARM	0.0695	0.6954	0.6701	0.1957	**0.037**	**0.7394**	0.6754	0.1212
	SASRec	0.0703	0.7033	0.6650	0.1942	0.0364	0.7276	0.6703	0.1209
FS-NYC	**BERT4Rec**	**0.0789**	**0.7895**	**0.6689**	**0.1909**	**0.0430**	**0.8596**	**0.6856**	**0.1230**
	CORE	0.0510	0.5097	0.4237	0.1206	0.0297	0.5937	0.4453	0.0785
	GRU4Rec	0.0752	0.7516	0.6409	0.1837	0.041	0.8199	0.6593	0.1179
	NARM	0.0338	0.338	0.2310	0.0629	0.0222	0.4432	0.2583	0.0445
	SASRec	0.0730	0.7304	0.6149	0.1760	0.0407	0.8135	0.6353	0.1137
FS-TKY	**BERT4Rec**	0.0816	0.8155	0.6721	0.1921	**0.0437**†	**0.8735**†	0.6861	0.1242
	CORE	0.0578	0.5778	0.4753	0.1351	0.0328	0.6568	0.4947	0.0882
	GRU4Rec	0.0772	0.7719	0.6259	0.1785	0.0422	0.8443	0.6448	0.1163
	NARM	0.0443	0.4431	0.3121	0.0858	0.0274	0.5473	0.3377	0.0595
	SASRec	**0.0820**†	**0.8203**†	**0.6896**†	**0.1981**†	0.0435	0.8692	**0.7023**†	**0.1274**†
ML-100k	**BERT4Rec**	0.0549	0.5493	0.3075	0.0785	0.0363	0.7253	0.3499	0.0604
	CORE	0.0352	0.3521	0.1886	0.0478	0.025	0.5005	0.2250	0.0378
	GRU4Rec	**0.0643**	**0.6426**	**0.3736**	**0.0986**	**0.0393**	**0.7869**	**0.4138**	**0.073**
	NARM	0.0273	0.2725	0.1411	0.0352	0.022	0.4401	0.1813	0.0291
	SASRec	0.0602	0.6023	0.3354	0.0861	0.0383	0.7667	0.3754	0.0659
ML-1M	**BERT4Rec**	0.0752	0.7517	0.4944	0.1357	0.0428	0.8561	0.5214	0.0949
	CORE	0.0457	0.4571	0.2601	0.0669	0.0312	0.6238	0.3029	0.0517
	GRU4Rec	**0.0788**†	**0.7876**†	**0.5614**†	**0.1572**†	**0.0438**†	**0.8758**†	**0.5837**†	**0.1065**†
	NARM	0.0428	0.4285	0.2398	0.0611	0.0303	0.6051	0.2840	0.0480
	SASRec	0.0761	0.7614	0.5063	0.1396	0.0431	0.8614	0.5319	0.0971
ML-20M	**BERT4Rec**	0.0942	0.9419	0.7381	0.2118	0.0489	0.9773	0.7471	0.1381
	CORE	0.0786	0.7863	0.5269	0.1460	0.0451	0.9019	0.5578	0.1012
	GRU4Rec	**0.0952**†	**0.9521**†	**0.7741**†	**0.2233**†	**0.0491**†	**0.9812**†	**0.7817**†	**0.1440**†
	NARM	0.0871	0.8706	0.6172	0.1736	0.0468	0.9355	0.6339	0.1172
	SASRec	0.0943	0.9432	0.7284	0.2086	0.0490	0.9807	0.7380	0.1366

performance. In this experiment, we set the embedding size to 50 while varying the sequence length over the range of values {20, 50, 100, 200}.

Interestingly, for the Beauty dataset, characterized by generally short sequences, there was no significant difference in NDCG@10 when changing the input sequence length. This suggests that the models were able to maintain performance even with excessive padding. This is a positive finding, demonstrating robustness to potential noise introduced by padding in shorter sequences.

Conversely, for all other datasets, attention-based models like SASRec, BERT4Rec, and GRU4Rec, the NDCG@10 increased along with the sequence length. This finding aligns with the intuition that these models can potentially capture richer user preferences with access to a longer history of interactions. Furthermore, it contradicts the claim made in the original SASRec paper [8] that the model does not scale effectively to very long sequences. Our results suggest that SASRec may benefit from longer sequences, prompting further investigation into this aspect.

NARM and CORE exhibited contrasting behaviour. In the ML-100k dataset, their NDCG@10 scores even decreased with increasing sequence length. This is noteworthy because the original CORE paper [21] only evaluated sequences up to 50 items, limiting insights into its behaviour with longer sequences. Conversely, the original NARM paper [19] claimed that NARM performs well in modeling long sessions, which aligns with our observations.

These findings highlight the importance of considering the model architecture and dataset characteristics when determining the optimal input sequence length for SRSs.

C. RQ2(B): EMBEDDING SIZE

A quick glance at the table could lead to the erroneous conclusion that GRU4Rec is generally the superior choice for an SRS. However, such a deduction would be deeply flawed.

Fig. 3 investigates how different embedding sizes – 32, 64, 128, 256, and 512 – affect various models.

One striking observation is the marked influence of embedding size on SASRec and BERT4Rec. As the embedding size increases, these models show a clear improvement in performance. This phenomenon suggests that their attention mechanisms effectively exploit higher-dimensional embeddings to refine input and output representations, leading to improved predictive capabilities.

In contrast, the performance of GRU-based models, i.e. GRU4Rec and NARM, is relatively unaffected by changes in the dimensionality of the embeddings. This indicates that changes in embedding size primarily affect the number of

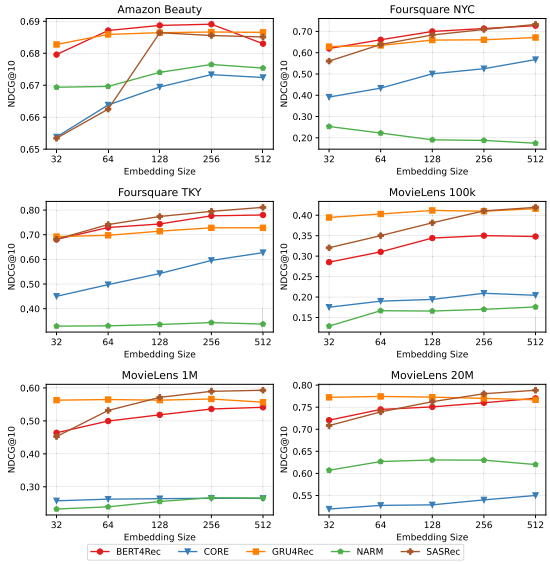


FIGURE 3. Effect of embedding size on model performance, as measured by NDCG@10. Each plot shows the results of the five models on one dataset.

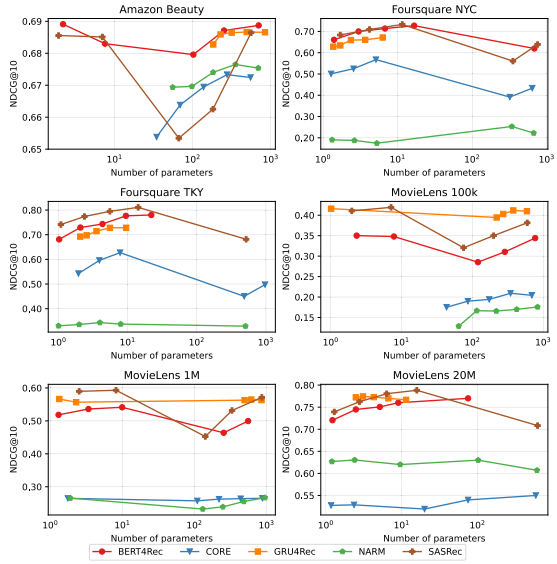


FIGURE 4. Effect of total number of model's parameters on the performance, as measured by NDCG@10. Each plot shows the results of the five models on one dataset.

parameters in the GRU layer without significantly impacting the computational capability.

CORE exhibits contrasting performance across different datasets. Remarkably, it demonstrates good performance on the Beauty and Foursquare datasets with increased embedding size. However, on MovieLens datasets, CORE displays no discernible increase in performance despite variations in the embedding size.

A notable divergence from the traditional model hierarchy emerges from this analysis: transformer-based models outperform GRU-based models as embedding size increases. This suggests that, given adequate computational resources, transformer-based models offer a compelling advantage at larger embedding sizes.

Interestingly, SASRec emerges as the best-performing model when using the largest embedding size, a finding not immediately apparent from Table 3. This highlights the nuanced relationship between model architecture, embedding size and performance.

These results highlight the importance of considering both model architecture and embedding size when designing recommendation systems, particularly in resource-rich environments where computational cost is not a limiting factor.

D. RQ3: MODEL SIZE

A fundamental aspect of conducting a truly fair comparison between models is to consider their respective sizes in terms of parameters. This ensures that any differences in performance are due to inherent architectural differences and not simply to the number of parameters. Fig. 4 shows a comparison based on this principle.

We see how the comparison becomes more nuanced when model size is considered. For example, while the number of parameters in GRU4Rec remains relatively stable even with increasing embedding sizes, transformer-based models show a contrasting trend. These models tend to grow in number of parameters with increasing embedding size more than the others.

For specific datasets, such as ML-100k and Beauty, the analysis shows that NARM still has room for improvement, as it maintains a relatively lower total number of parameters compared to other models. This suggests that NARM may have untapped potential to improve performance without significantly increasing model complexity, making it an interesting avenue for further exploration and optimisation. Regarding the Foursquare datasets, the number of parameters for NARM increases significantly when changing the embedding size, while the results in terms of NDCG remain approximately the same.

E. RQ4: EMISSIONS

In addition to evaluating model performance, we assess the environmental footprint of each model to identify potential trade-offs between performance and environmental impact. Our aim is to shed light on the sustainability implications associated with different model configurations.

Fig. 5 visualises the performance as a function of the environmental impact, described by CO_2-eq emissions in kg. A notable trend across all models is a positive correlation between CO_2-eq emissions and performance. While SASRec and BERT4Rec exhibit superior performance to GRU4Rec, particularly at larger embedding sizes as discussed in section V-C, they also have a generally higher environmental impact. Transformer-based models are known for their high computational requirements, driven by their large parameter sizes [40]. This enhanced computational demand poses challenges in terms of hardware resources

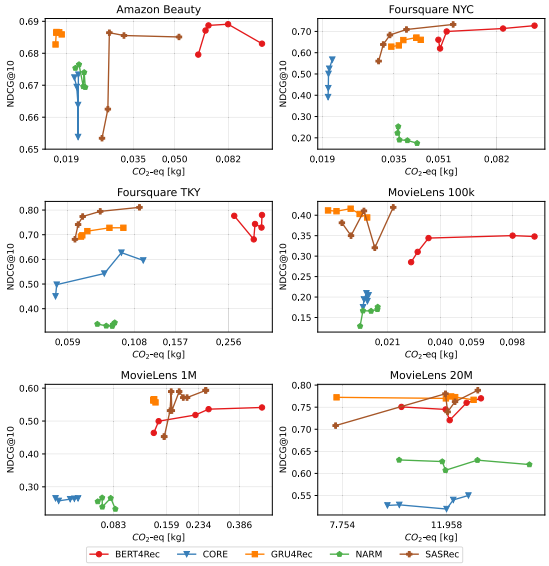


FIGURE 5. Relation between emissions, measured as CO_2-eq in Kg, and performance, measured by NDCG@10. Each plot shows the results of the five models on one dataset.

and raises concerns regarding energy consumption and environmental sustainability. In addition, it should be noted that the consumption for ML-20M is at least 10 times higher than all other datasets.

After careful consideration, the optimal compromise appears to be the use of SASRec with an embedding size of 256. This choice strikes a balance between achieving competitive NDCG@10 values and maintaining energy consumption levels comparable to other models. This finding highlights the importance of considering environmental sustainability alongside performance metrics when selecting and optimising recommendation models.

We advocate that the recommender systems community to adopt a more unified and consistent approach to assessing the environmental impact of its experiments [41]. While some initiatives in this direction have begun, they remain fragmented and lack standardization. A dedicated library that natively and intuitively calculates the environmental footprint of experiments would be a significant advancement. We believe that our framework takes a meaningful step toward addressing this gap, promoting more sustainable practices in SRS research and development.

VI. CONCLUSION

Our contribution addresses reproducibility concerns in Sequential Recommendation research by introducing a standardized pre-processing and training methodology for SRSs, along with a user-friendly framework, EasyRec, for creating custom models with minimal coding effort. Our experimental results challenge existing literature by demonstrating the effectiveness of GRU4Rec with low-dimensional embeddings and the superiority of transformer-based models with larger embedding sizes. We also highlight the importance of managing input sequence length based on dataset statistics. To ensure fair model comparison, we introduce an analysis based on the number of model parameters and conduct an energy consumption analysis, identifying a trade-off between sustainability and performance. Our work represents a step towards standardizing benchmarks and pre-processing methods, promoting consistency in future research. We plan to extend our code resource with additional models, datasets, and insights to contribute to the ongoing evolution of SRS research.

A. LIMITATIONS AND FUTURE IMPROVEMENTS

As a relatively young library, our framework has some limitations, including a limited number of datasets, models, and general functionalities. However, as detailed in section IV, the library has been designed with extensibility in mind, making it easy to incorporate additional features. We expect these gaps to be filled through ongoing efforts by us, as well as contributions from the broader research community.

Currently, the library places a strong emphasis on sequential recommendation, with limited support for other techniques such as matrix factorization, convolutional approaches, and graph-based methods. This focus is consistent with the primary scope of this work, which targets the sequential domain. Nevertheless, some initial implementations of these techniques are already available in the library and are being actively extended. We envision that future iterations of the library will evolve into a more comprehensive resource that addresses a wider range of recommendation paradigms.

ACKNOWLEDGMENT

(Filippo Betello, Antonio Purificato, and Federico Siciliano contributed equally to this work.) The work of Andrea Bacciu was done before joining Amazon.

REFERENCES

[1] B. Smith and G. Linden, "Two decades of recommender systems at Amazon.com," *IEEE Internet Comput.*, vol. 21, no. 3, pp. 12–18, May 2017.

[2] M. Quadrana, P. Cremonesi, and D. Jannach, "Sequence-aware recommender systems," *ACM Comput. Surv.*, vol. 51, no. 4, pp. 1–36, Jul. 2019.

[3] S. Rendle, C. Freudenthaler, and L. Schmidt-Thieme, "Factorizing personalized Markov chains for next-basket recommendation," in *Proc. 19th Int. Conf. World Wide Web*. New York, NY, USA: Association for Computing Machinery, Apr. 2010, pp. 811–820, doi: 10.1145/1772690.1772773.

[4] S. Wang, L. Hu, Y. Wang, L. Cao, Q. Z. Sheng, and M. Orgun, "Sequential recommender systems: Challenges, progress and prospects," in *Proc. 28th Int. Joint Conf. Artif. Intell.*, Aug. 2019, pp. 6332–6338, doi: 10.24963/ijcai.2019/883.

[5] K. Cho, B. van Merrienboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio, "Learning phrase representations using RNN encoder–decoder for statistical machine translation," 2014, *arXiv:1406.1078*.

[6] B. Hidasi, A. Karatzoglou, L. Baltrunas, and D. Tikk, "Session-based recommendations with recurrent neural networks," 2016, *arXiv:1511.06939*.

[7] F. Sun, J. Liu, J. Wu, C. Pei, X. Lin, W. Ou, and P. Jiang, "BERT4Rec: Sequential recommendation with bidirectional encoder representations from transformer," in *Proc. 28th ACM Int. Conf. Inf. Knowledge Manage.*, 2019, pp. 1441–1450.

[8] W.-C. Kang and J. McAuley, "Self-attentive sequential recommendation," in *Proc. IEEE Int. Conf. Data Mining (ICDM)*, Nov. 2018, pp. 197–206.

[9] M. Ferrari Dacrema, P. Cremonesi, and D. Jannach, "Are we really making much progress? A worrying analysis of recent neural recommendation approaches," in *Proc. 13th ACM Conf. Recommender Syst.*, Sep. 2019, pp. 101–109.

[10] Z. Sun, D. Yu, H. Fang, J. Yang, X. Qu, J. Zhang, and C. Geng, "Are we evaluating rigorously? Benchmarking recommendation for reproducible evaluation and fair comparison," in *Proc. 14th ACM Conf. Recommender Syst.*, Sep. 2020, pp. 23–32.

[11] M. Ferrari Dacrema, P. Cremonesi, and D. Jannach, "Methodological issues in recommender systems research," in *Proc. Int. Joint Conf. Artif. Intell.*, 2020, pp. 4706–4710.

[12] M. Ferrari Dacrema, S. Boglio, P. Cremonesi, and D. Jannach, "A troubling analysis of reproducibility and progress in recommender systems research," *ACM Trans. Inf. Syst.*, vol. 39, no. 2, pp. 1–49, Apr. 2021.

[13] P. Cremonesi and D. Jannach, "Progress in recommender systems research: Crisis? What crisis?" *AI Mag.*, vol. 42, no. 3, pp. 43–54, Sep. 2021.

[14] B. Hidasi and Á. T. Czapp, "The effect of third party implementations on reproducibility," in *Proc. 17th ACM Conf. Recommender Syst.*, Sep. 2023, pp. 272–282.

[15] B. Hidasi and Á. T. Czapp, "Widespread flaws in offline evaluation of recommender systems," in *Proc. 17th ACM Conf. Recommender Syst.*, Sep. 2023, pp. 848–855.

[16] A. Petrov and C. Macdonald, "A systematic review and replicability study of BERT4Rec for sequential recommendation," in *Proc. 16th ACM Conf. Recommender Syst.*, Sep. 2022, pp. 436–447.

[17] F. Betello, F. Siciliano, P. Mishra, and F. Silvestri, "Investigating the robustness of sequential recommender systems against training data perturbations," in *Advances in Information Retrieval*, N. Goharian, N. Tonellotto, Y. He, A. Lipani, G. McDonald, C. Macdonald, and I. Ounis, Eds., Cham, Switzerland: Springer, 2024, pp. 205–220.

[18] A. Beutel, P. Covington, S. Jain, C. Xu, J. Li, V. Gatto, and E. H. Chi, "Latent cross: Making use of context in recurrent recommender systems," in *Proc. 11th ACM Int. Conf. Web Search Data Mining*. New York, NY, USA: Association for Computing Machinery, Feb. 2018, pp. 46–54, doi: 10.1145/3159652.3159727.

[19] J. Li, P. Ren, Z. Chen, Z. Ren, T. Lian, and J. Ma, "Neural attentive session-based recommendation," in *Proc. ACM Conf. Inf. Knowl. Manage.* New York, NY, USA: Association for Computing Machinery, Nov. 2017, pp. 1419–1428, doi: 10.1145/3132847.3132926.

[20] J. Devlin, M. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. Conf. North American Chapter Assoc. Comput. Linguistics, Human Lang. Technol.*, vol. 1, J. Burstein, C. Doran, and T. Solorio, Eds., Minneapolis, Minnesota: Association for Computational Linguistics, Jun. 2018, pp. 4171–4186. [Online]. Available: https://aclanthology.org/N19-1423

[21] Y. Hou, B. Hu, Z. Zhang, and W. X. Zhao, "CORE: Simple and effective session-based recommendation within consistent representation space," in *Proc. 45th Int. ACM SIGIR Conf. Res. Develop. Inf. Retr.*, Jul. 2022, pp. 1796–1801.

[22] M. Li, A. Vardasbi, A. Yates, and M. de Rijke, "Repetition and exploration in sequential recommendation," in *Proc. 46th Int. ACM SIGIR Conf. Res. Develop. Inf. Retr.*, 2023, pp. 2532–2541.

[23] W. X. Zhao, S. Mu, Y. Hou, Z. Lin, Y. Chen, X. Pan, K. Li, Y. Lu, H. Wang, C. Tian, Y. Min, Z. Feng, X. Fan, X. Chen, P. Wang, W. Ji, Y. Li, X. Wang, and J.-R. Wen, "RecBole: Towards a unified, comprehensive and efficient framework for recommendation algorithms," in *Proc. 30th ACM Int. Conf. Inf. Knowledge Manage.*, 2021, pp. 4653–4664.

[24] A. Klenitskiy and A. Vasilev, "Turning dross into gold loss: Is BERT4Rec really better than SASRec?" in *Proc. 17th ACM Conf. Recommender Syst.*, Sep. 2023, pp. 1120–1125.

[25] J. Lin, "The neural hype and comparisons against weak baselines," in *Proc. ACM SIGIR Forum*, New York, NY, USA, 2019, pp. 40–51.

[26] M. Ludewig and D. Jannach, "Evaluation of session-based recommendation algorithms," *User Model. User-Adapted Interact.*, vol. 28, nos. 4–5, pp. 331–390, Dec. 2018.

[27] J. McAuley, C. Targett, Q. Shi, and A. van den Hengel, "Image-based recommendations on styles and substitutes," in *Proc. 38th Int. ACM SIGIR Conf. Res. Develop. Inf. Retr.* New York, NY, USA: Association for Computing Machinery, Aug. 2015, pp. 43–52, doi: 10.1145/2766462.2767755.

[28] D. Yang, D. Zhang, V. W. Zheng, and Z. Yu, "Modeling user activity preference by leveraging user spatial temporal characteristics in LBSNs," *IEEE Trans. Syst., Man, Cybern., Syst.*, vol. 45, no. 1, pp. 129–142, Jan. 2015.

[29] F. M. Harper and J. A. Konstan, "The MovieLens datasets: History and context," *ACM Trans. Interact. Intell. Syst.*, vol. 5, no. 4, pp. 1–19, Dec. 2015, doi: 10.1145/2827872.

[30] R. Pascanu, T. Mikolov, and Y. Bengio, "On the difficulty of training recurrent neural networks," 2013, *arXiv:1211.5063*.

[31] I. J. Good, "Rational decisions," *J. Roy. Stat. Soc., Ser. B*, vol. 14, no. 1, pp. 107–114, Jan. 1952.

[32] D. B. Hunter, J. E. Salzman, and D. Zaelke, "Glasgow climate summit: COP26," UCLA School Law, Public Law Res. Paper 22-02, 2021. [Online]. Available: https://scholar.googleusercontent.com/scholar.bib?q=info:aHlqZL6AyNYJ:scholar.google.com/&output=citation&scisdr=ClHi-Ji1EKeQgMiiVzk:AFWwaeYAAAAAZ32kTzmnUlP_lfTmQpD8-L_FOjo&scisig=AFWwaeYAAAAAZ32kT-LClZmBcgVTJ9kNIjevtsY&scisf=4&ct=citation&cd=-1&hl=it.

[33] D. Patterson, J. Gonzalez, Q. Le, C. Liang, L.-M. Munguia, D. Rothchild, D. So, M. Texier, and J. Dean, "Carbon emissions and large neural network training," 2021, *arXiv:2104.10350*.

[34] B. Courty et al., "MLCO2/codecarbon: V2.3.2," Tech. Rep., Nov. 2023, doi: 10.5281/zenodo.10213072. [Online]. Available: https://github.com/mlco2/codecarbon

[35] B. Kim and R. Neff, "Measurement and communication of greenhouse gas emissions from US food consumption via carbon calculators," *Ecological Econ.*, vol. 69, no. 1, pp. 186–196, Nov. 2009.

[36] V. W. Anelli, A. Bellogin, A. Ferrara, D. Malitesta, F. A. Merra, C. Pomo, F. M. Donini, and T. Di Noia, "Elliot: A comprehensive and rigorous framework for reproducible recommender systems evaluation," in *Proc. 44th Int. ACM SIGIR Conf. Res. Develop. Inf. Retr.*, Jul. 2021, pp. 2405–2414.

[37] A. Salah, Q. Truong, and H. W. Lauw, "Cornac: A comparative framework for multimodal recommender systems," *J. Mach. Learn. Res.*, vol. 21, no. 95, pp. 1–5, Jan. 2020.

[38] L. Michiels, R. Verachtert, and B. Goethals, "RecPack: An(other) experimentation toolkit for top-N recommendation using implicit feedback data," in *Proc. 16th ACM Conf. Recommender Syst.* New York, NY, USA: Association for Computing Machinery, Sep. 2022, pp. 648–651, doi: 10.1145/3523227.3551472.

[39] C. Wang, M. Zhang, W. Ma, Y. Liu, and S. Ma, "Make it a chorus: Knowledge- and time-aware item modeling for sequential recommendation," in *Proc. 43rd Int. ACM SIGIR Conf. Res. Develop. Inf. Retr.*, Jul. 2020, pp. 109–118.

[40] E. M. Bender, T. Gebru, A. McMillan-Major, and S. Shmitchell, "On the dangers of stochastic parrots: Can language models be too big?" in *Proc. ACM Conf. Fairness, Accountability, Transparency*. New York, NY, USA: Association for Computing Machinery, Mar. 2021, pp. 610–623, doi: 10.1145/3442188.3445922.

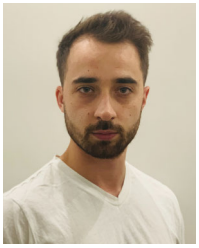
[41] A. Purificato and F. Silvestri, "Eco-aware graph neural networks for sustainable recommendations," 2024, *arXiv:2410.09514*.

FILIPPO BETELLO received the master's degree in artificial intelligence and robotics from the Sapienza University of Rome, where he is currently pursuing the Ph.D. degree in data science with the Department of Computer, Control and Management Engineering "Antonio Ruberti." He is working under the guidance of Prof. Fabrizio Silvestri and collaborating with the RSTless Group. His research interests include recommender systems, information retrieval, and LLMs.

ANTONIO PURIFICATO received the master's degree in artificial intelligence and robotics from the Sapienza University of Rome, where he is currently pursuing the Ph.D. degree in data science with the Department of Computer, Control and Management Engineering "Antonio Ruberti." He is working under the guidance of Prof. Fabrizio Silvestri and collaborating with the RSTless Group. His research interests include graph neural networks and category theory, with a focus on the environmental impact of deep learning algorithms.

FEDERICO SICILIANO received the Ph.D. degree in data science, in 2024. His Ph.D. thesis was on architectural components of trustworthy artificial intelligence with Prof. Fabrizio Silvestri. He is a Postdoctoral Researcher with the RSTLess Labortory, Sapienza University of Rome, under the guidance of Prof. Fabrizio Silvestri. He has the privilege of partnering with several world-renowned institutions, including Cambridge University, Meta, Amazon, and UniPi. His current research projects include information retrieval, recommender systems, and explainable artificial intelligence.

GIOVANNI TRAPPOLINI received the Ph.D. degree in data science, in 2022. His Ph.D. thesis was on geometric deep learning, under the supervision of Prof. Emanuele Rodolà. He is an Assistant Professor with the Sapienza University of Rome. He has the privilege of partnering with several world-renowned institutions, including Stanford, Technion, Meta, Amazon, TII, and UniPi. His current research project lies at the intersection of multimodal deep learning and information retrieval. In particular, he has been experimenting with RAG models.



ANDREA BACCIU received the master's degree (Hons.) in computer science from the Sapienza University of Rome, where he is currently pursuing the Ph.D. degree in natural language processing, under the supervision of Prof. Fabrizio Silvestri and Prof. Nicola Tonellotto. He has presented a research work on cross-lingual semantic role labeling, which was published and received the Outstanding Long Paper at the NAACL 2021 Conference. His research interests include large language models, natural language understanding, and information retrieval.

NICOLA TONELLOTTO is an Associate Professor with the Department of Information Engineering, University of Pisa. From 2002 to 2019, he was a Researcher with the Information Science and Technologies Institute "A. Faedo' of the National Research Council of Italy. He is an Honorary Research Fellow with the College of Science and Engineering, School of Computing Science, University of Glasgow, since 2020. His main research interests include cloud computing, web search, and information retrieval, with a particular focus on efficient data processing and neural information retrieval. He has co-authored more than 100 papers on these topics in peer-reviewed international journals and conferences. He has a Distinguished Member of ACM, since 2023.

FABRIZIO SILVESTRI (Member, IEEE) received the Ph.D. degree in computer science from the University of Pisa. He is a Full Professor and the Coordinator of the Ph.D. degree in data science with the Dipartimento di Ingegneria informatica, automatica e gestionale (DIAG), Sapienza University of Rome. He spent eight years abroad in industrial research laboratories (Yahoo! and Facebook). At Facebook AI, he has directed research groups to develop artificial intelligence techniques to combat malicious actors, who use the Facebook platform for malicious purposes (hate speech, misinformation, and terrorism). He is the author of more than 150 papers in international journals and conference proceedings. He holds nine industrial patents. His research interests include artificial intelligence, and in particular, he deals with machine learning applied to web search problems and natural language processing. He is the holder of the "test-of-time" Award at the ECIR 2018 Conference for an article published in 2007. He is the holder of three best paper awards and other international awards.

Open Access funding provided by 'Università degli Studi di Roma "La Sapienza" 2' within the CRUI CARE Agreement